



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Comparison of Stencil2D Performance: NumPy, Numba, Torch, JAX and GT4Py

High Performance Computing for Weather and Climate

Emilia Clauss, Nora Joss, Mario Wildhaber

Institute for Atmospheric and Climate Science
Department Environmental Systems Science, ETH Zürich

Supervisors:

Oliver Fuhrer

September 13, 2026

Summary

This report benchmarks Python frameworks (NumPy, Numba, CuPy, PyTorch, JAX, GT4Py) against a Fortran reference for stencil computations in weather modeling, evaluating both performance and implementation complexity. Results show that while JAX on GPU delivers the highest asymptotic performance and robustness across domain shapes, simpler tools like CuPy and Numba might be the better choice for simple projects in which higher implementation costs are not worth it. The study concludes that the optimal framework depends on the specific hardware availability and the willingness to refactor code, with no single solution dominating all scenarios.

Contents

Summary	ii
1. Introduction	4
2. Methods	4
2.1. Hardware	5
2.2. Python-based frameworks	5
2.3. Benchmark Protocol	7
2.4. Initial Field and Validation	7
2.5. Additional diagnostics	8
2.6. Validation of Implementation Experience	9
3. Results & Discussion	10
3.1. Benchmarking experiments	10
3.2. Field Validation	15
3.3. Additional Diagnostics	15
3.4. Implementation Experience	17
4. Conclusion	19
4.1. Performance	19
4.2. Choice of Framework / DSL	19
5. Outlook	20
References	21
A. Library Versions	22
B. Additional Figures	27
C. Use of AI-Based Tools	30

1. Introduction

Stencil computations play a crucial role in high performance computing (HPC) for weather and climate models, particularly in solving time-dependent partial differential equations (PDEs) such as those arising in fluid dynamics or heat diffusion. A stencil is a computational pattern that updates the value of a grid cell based on its own value and the values of neighbouring grid points. The same stencil pattern is applied uniformly across the entire computational grid. In this report, a five-point finite difference stencil is used to approximate the discretized Laplacian on a two-dimensional grid. The Laplacian is applied twice to the grid field, resulting in a fourth-order diffusion scheme known as *biharmonic diffusion operator*. For each grid point (i, j) , the Laplacian is approximated by subtracting four times the value of the centre cell from the sum of its four direct neighbours, divided by the grid spacing squared. The grid spacing Δx is here set to $x = 1$, allowing the factor $\frac{1}{\Delta x^2}$ to be omitted by the Laplacian operator.

$$\Delta_h \phi_{i,j}^n \approx \frac{-4\phi_{i,j}^n + \phi_{i-1,j}^n + \phi_{i+1,j}^n + \phi_{i,j-1}^n + \phi_{i,j+1}^n}{\Delta x^2} \quad (1.1)$$

Time is discretized using a simple forward Euler scheme:

$$\frac{\partial \phi_{i,j}^n}{\partial t} \approx \frac{\phi_{i,j}^{n+1} - \phi_{i,j}^n}{\Delta t} \quad (1.2)$$

Combining the time discretization with the *biharmonic diffusion operator* yields the implemented diffusion step, where α is a stability parameter that is tuned accordingly:

$$\phi_{i,j}^{n+1} = \phi_{i,j}^n - \alpha \Delta_h (\Delta_h \phi_{i,j}^n) \quad (1.3)$$

To ensure a working five-point stencil at the boundaries of the two-dimensional grid, two gridcells of halo points (ghost cells) are introduced. These halo points are updated periodically, meaning that values from the opposite edge of the domain are copied into the halo region, allowing grid points to be influenced by not only their immediate neighbours but also by cells across the entire domain.

Stencil computations are computationally expensive, and optimization is essential for large-scale weather and climate applications. This report discusses and compares several Python-based frameworks that can be used to optimize the stencil computation described above. The evaluation considers not only performance improvements but also the applicability of each framework, trying to identify the best balance between efficiency and implementation complexity for stencil computations.

2. Methods

2.1. Hardware

The computation is executed on the Alps supercomputer at the Swiss National Supercomputer Centre (CSCS), using the Santis cluster. Santis uses NVIDIA Grace-Hopper (GH200) nodes, combining a Grace CPU with a Hopper GPU [2]. All computations were limited to the use of a single node.

2.2. Python-based frameworks

The following python-based languages are used to approximate the fourth-order diffusion scheme on a two-dimensional grid. Whenever possible the calculations are conducted on Central Processing Units (CPUs) and Graphic Processing Units (GPUs).

NumPy

NumPy is a python library used for numerical computation. It allows an implementation of the 2D stencil computation using array slicing and vectorization, and avoiding nested for-loops. This leads to a speed up, as additions can be made across the entire sub-grid in parallel memory operations, instead of looping through each element [5].

For our 2D stencil we used vectorized slicing for both the Laplacians and the halo update, and used the version developed during the course.

CuPy

CuPy is a python library for GPU accelerated numerical computing. It provides a similar interface to NumPy, allowing an easy transfer from existing NumPy code to run on GPUs. CuPy uses CUDA to perform numerical operations on NVIDIA GPUs [3].

For our 2D stencil implementation we used the CuPy code developed during the course.

Numba

Numba is a python library that allows for major speed-ups with little additional coding work. By simply adding some decorators the code becomes much faster. Numba does this by implementing a just in time (JIT) compilation approach. With this, Numba compiles everything to machine code first before running it, instead of using the default (and slow) Python-interpreter. It also

2. Methods

parallelizes loops if possible. However this also brings a newly added overhead: The compilation time for the machine code.[4] In the 2D stencil approach done here the default python with individual loops for each dimension was used. By adding `@jit(parallel=True, fastmath=True, cache=True)` as a decorator above the individual functions Numba with parallelization and some further optimizations (caching and fast math) could be activated.

PyTorch

PyTorch is an machine learning framework, developed for numerical computation and deep learning. It provides tensors, which are similar to NumPy arrays but can efficiently perform computations on both CPUs and GPUs. Moreover are the tensors optimized for automatic differentiation, which is essential for training neural networks [6].

In our 2D stencil implementation we changed the NumPy arrays to tensors, and to reduce computational and memory overhead, gradient tracking was disabled using `torch.no_grad()`, as it is not required for the stencil calculation. For accurate GPU runtime measurements, `torch.cuda.synchronize()` was used before and after the timed operation to ensure that all GPU computations had completed. The 2D stencil implementation was evaluated on both CPU and GPU.

JAX

The python library JAX allows for array-based numerical computations with a NumPy inspired interface. Additionally, it provides multiple features that make it particularly efficient for a variety of tasks, like automatic differentiation transformations, the automatic function vectorization option or the built-in just-in-time (JIT) compiler based on Open XLA. Moreover it runs on both CPU and GPU. [1]

We use the JIT compilation feature to improve the runtime performance of our 2D stencil implementation and test it both on CPU and GPU. While the procedure and code structure is the same as for the other implementations, a few adjustments have to be made in the JAX version. JIT compilation is enabled with the `@partial(jax.jit)` decorator and for-loops are rewritten with `lax.fori_loop` to allow for efficient use of the JIT compiler. Because JAX arrays are immutable, passing array values by slicing is not possible; instead, we use the construction `.at[].set()`. The functions `laplacian`, `update_halo` and `apply_diffusion` are slightly adapted to make them functionally pure, and to account for the asynchronous dispatch JAX uses to reduce Python overhead, we use `.block_until_ready()` when measuring execution times. Finally, to allow for the choice of GPU or CPU as device, and additional `@click.option` is added.

GT4Py

GT4Py stands for Grid-Tools for Python and is a domain specific language compiler developed for climate and weather modelling applications. It allows to implement stencils in GTScript using standard Python syntax and then generates an optimized C++ code which with JIT compilation reaches very high performances. GT4Py codes can be executed on both CPU and GPU architectures. Here, we use the version developed during the course.

2.3. Benchmark Protocol

We benchmark the stencil implementations by measuring the execution time require to apply the diffusion operator for a fixed number of iterations (1024 in our experiments). The benchmarks are performed on domains with horizontal dimensions ranging from 16 to 512 grid cells and comprising 16, 32 or 64 vertical levels. All combinations of the specified horizontal grid sizes are considered. Each configuration is executed 5 times and the average runtime across the five runs is used as performance metric. To assess the performance gains achievable by switching from CPU to GPU execution, the supported frameworks are evaluated on both backends, with the exception of GT4Py which is only benchmarked on CPU. The benchmarking workflow is managed through the `master.py` script, while the different configurations are specified in the `config.yaml` file.

2.4. Initial Field and Validation

All stencil implementations are initialized with the same field, which is then evolved through the diffusion operator to the final field, shown in Figure 2.1. The initial field is divided into four quadrants, each containing a different pattern. One quadrant contains a single large spike, another contains several smaller spikes, the third is filled with random values, and the fourth is filled with zeros. This choice of initial field provides several distinct features that can be used to detect implementation errors. In particular, the different patterns make it possible to identify incorrect halo updates or errors in the diffusion operator. Furthermore, the evolution of the field allows potential numerical stability issues to be identified, for example through unexpected growth or oscillation in the field values.

2. Methods

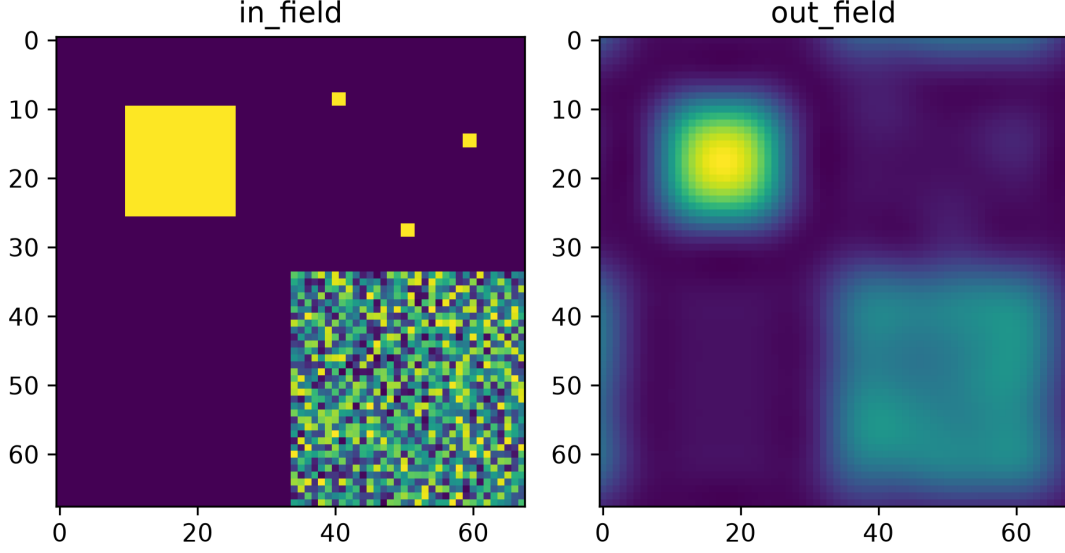


Figure 2.1.: Initial (left) and final (right) fields produced by the Fortran implementation on a $64 \times 64 \times 64$ grid after 1024 iterations.

The resulting fields from the 2D stencil implementation are validated both visually and numerically against the output of the Fortran 2D stencil implementation. First, the resulting fields are inspected visually to verify that the initial patterns are smoothed as expected under diffusion and that the halo regions exhibit the expected values after the halo update. Second, the numerical results produced by each framework are compared with the Fortran reference implementation using the `np.allclose()` function. This comparison is performed by the `validation.py` script. Two fields are considered numerically equivalent if all corresponding grid cell values satisfy the absolute and relative tolerance of $1e^{-4}$ specified by `np.allclose()`.

2.5. Additional diagnostics

Besides the performance measurements automated in the `master.py` script, some additional tests are performed manually.

2. Methods

To investigate whether and how the different frameworks exploit multi-threading, we compare their performance under the default threading settings with runs where Open MP is restricted to a single thread (with `export OMP_NUM_THREADS=1`). To ensure comparability, the subsequent benchmarking is again done via the `master.py` script.

Additionally, we assess the overhead introduced by the `master.py` benchmarking script by comparing the measured runtimes against those obtained from manually executed benchmark runs. Here, only a small number of representative configurations is tested, as the objective of the test is just to provide an exploratory and qualitative assessment. Specifically, the manual runs are performed only for the cubic domain with 64 grid cells per side.

2.6. Validation of Implementation Experience

To determine which Pthon-based framework is easiest to apply to and implement our original 2D stencil NumPy code, we compare the number of lines of code that need to be changed or added during the implementation. We also evaluate whether the existing logic of the Laplacian operator and the halo updates can be retained or needs to be modified for each framework. Additionally, we consider our personal experiences when working with the different frameworks, including the ease of implementation and overall usability. Based on both the objective and subjective results, we aim to identify the Python-based framework that is easiest to apply to our existing NumPy 2D stencil code.

3. Results & Discussion

3.1. Benchmarking experiments

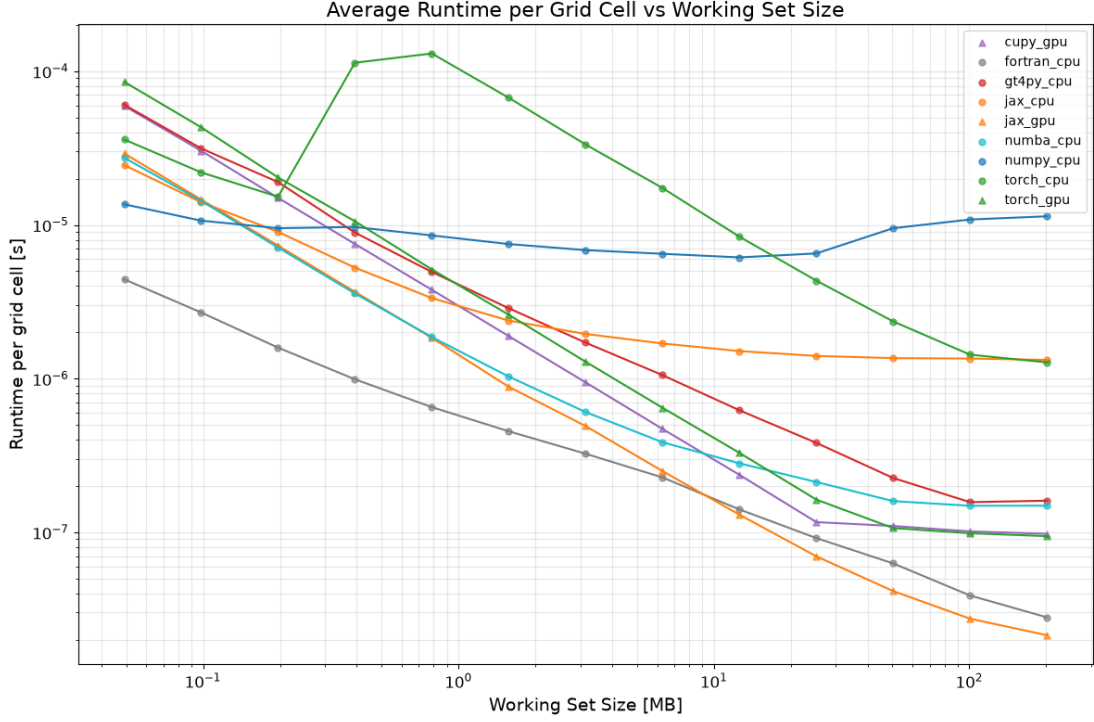


Figure 3.1.: Runtimes per grid cell as a function of the working set size. The working set size is given by $3 \cdot 4 \cdot n_x \cdot n_y \cdot n_z$ bytes, as 32-bit floating-point numbers were used. Different combinations of grid sizes in the three spatial dimensions can correspond to the same working set size. For each working set size, the plotted value represents the mean runtime over all corresponding grid-size combinations.

In general, the runtime per grid cell decreases as the working-set size increases as shown in Figure 3.1. This behaviour can largely be attributed to fixed overheads being amortized over an increasing number of grid cells. Domain shape may also influence performance through cache effects and differences in the perimeter-to-area ratio. However, because all domain shapes corresponding to the same working-set size are averaged together, the influence of shape cannot be isolated from this figure alone, and is discussed in more detail below.

The curve corresponding to the NumPy implementation does not show this decrease but is remarkably flat, meaning that the runtime grows approximately linearly with the number of grid

3. Results & Discussion

cells. This indicates that the NumPy implementation reaches its memory bandwidth limited limit very quickly due to inefficient memory usage.

The PyTorch implementation on CPU displays a behaviour that strongly deviates from the other CPU frameworks, exhibiting a pronounced increase in runtime per grid cell between approximately 0.1 and 1 MB, before performance increases again. The origin of this hump-shaped curve is not clear. One possible explanation is a change in the internal execution strategy, for example due to thread scheduling or kernel selection, as the problem size increases.

The Numba and GT4Py CPU implementations achieve very similar performance. Numba performs slightly better for smaller working-set sizes, whereas GT4Py closes the gap as the working-set size increases. Both implementations appear to approach a plateau for the largest working sets, suggesting that they are approaching hardware-imposed limits.

The JAX CPU implementation also benefits from increasing problem size, but its improvement is less pronounced and it stabilizes at a higher runtime per grid cell than Numba and GT4Py. This behaviour could be related to additional overhead, associated for example with the XLA compilation.

The Fortran version was also included in the benchmarking, despite serving mainly as ground truth for the validation of the field resulting after the application of the diffusion operator. It consistently delivers the lowest runtime per grid cell of all CPU implementations and maintains its advantage across the whole range of working-set sizes.

On GPU, all three implementations strongly benefit from increasing working set sizes, showing the ability of the GPU backends to exploit the large degree of parallelism available in the stencil computation once the problem size is sufficiently large. This is evident also in Figure 3.2, which shows that, while GPU execution is slower than CPU execution for small working-set sizes due to kernel-launch and data-management overheads, it provides a substantial speed-up for larger domains. The particularly sharp increase in speed-up observed for PyTorch is primarily caused by the peculiar behaviour of the corresponding CPU implementation discussed above, rather than by an exceptional improvement in GPU performance within the same range. JAX achieves the best asymptotic performance and ultimately becomes the fastest implementation overall, while CuPy and PyTorch are on similar performance levels.

3. Results & Discussion

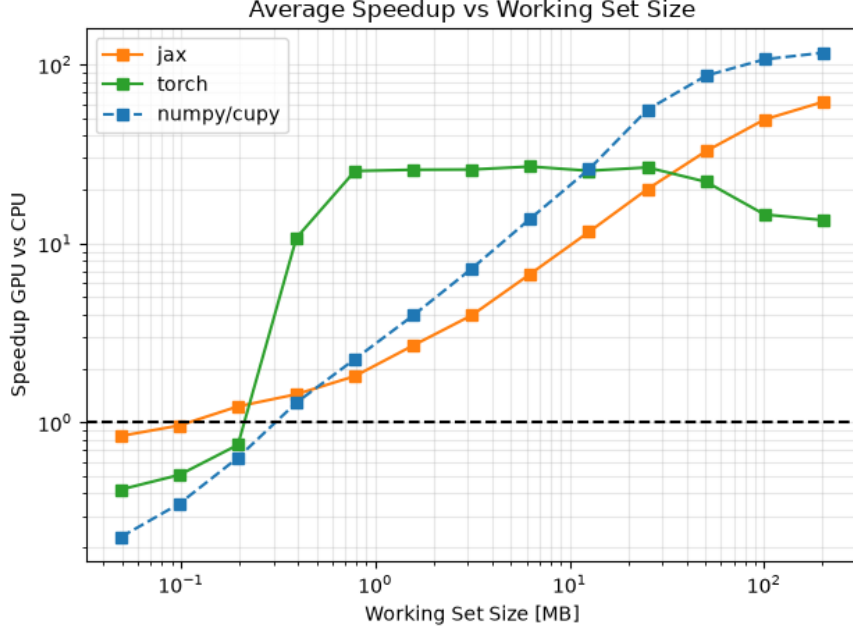


Figure 3.2.: Ratio of the runtime per grid cell on GPU relative to CPU (the effective "speed-up" achieved by switching to GPU) as a function of working-set size. Results are shown for all frameworks supporting both CPU and GPU backends, as well as for CuPy relative to NumPy. As in Figure 3.1, the plotted values represent averages over all domain shapes corresponding to the same working-set size.

3. Results & Discussion

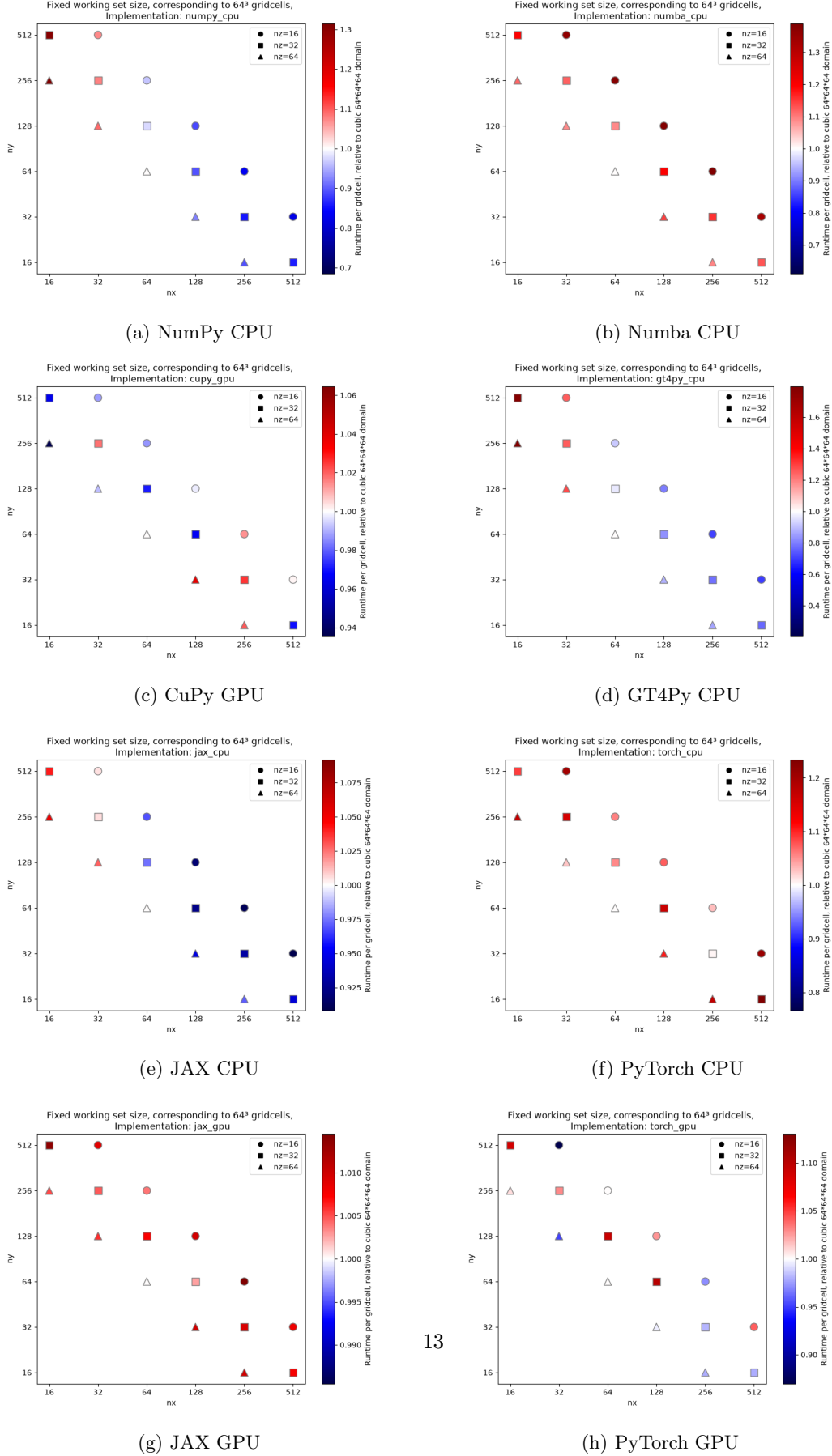


Figure 3.3.: Run time per grid cell for different domain shapes corresponding to the same working set size as a cube with 64 grid cells per side, relative to the runtime per grid cell for the cubic domain.

3. Results & Discussion

As already mentioned, the benchmarking experiments were performed for multiple different combinations of grid sizes in the three spatial dimensions. While in the results discussed up to now all these different combinations and configurations were averaged to obtain a single mean runtime per grid cell for each working-set size, Figure 3.3 shows the effect of the domain shape on the stencil code performance for a specific working set size and the different frameworks considered. In all plots, the working set is kept approximately constant and corresponds to the cubic reference domain of 64^3 . Consequently, changing one of the dimensions requires the other dimensions to be adjusted such that the total number of grid cells remains approximately constant. The x - and y -axes represent the dimensions n_x and n_y , respectively, while different marker shapes represent the different values of n_z . The colour of each marker indicates the runtime per grid cell relative to the cubic 64^3 reference case. Thus, a value of one corresponds to the performance of the 64^3 domain, while values larger or smaller than one indicate a higher or lower runtime per grid cell, respectively.

The results show that performance is not determined only by the volume of data processed, but is strongly influenced by domain geometry. This sensitivity is most pronounced across the CPU backends. NumPy CPU and Numba CPU have both an increased runtime per grid cells when the contiguous dimension n_x is small, as strided memory access reduces cache-line efficiency and loop vectorization, though performance recovers as n_x increases. GT4Py CPU displays the largest shape sensitivity of all implementations, with runtimes ranging from under $0.4\times$ to over $1.6\times$ the baseline, small n_x dimensions degrade its performance, but larger horizontal extents ($n_x, n_y \geq 128$) unlock massive speed-ups, showing that its generated C++ code heavily favours broad horizontal planes. JAX CPU and PyTorch CPU exhibit more moderate dependence, with JAX keeping variations bounded between $\approx 0.92\times$ and $1.08\times$ via XLA optimizations, and PyTorch showing localized slowdowns primarily when n_x is small.

In contrast, the GPU implementations display substantially higher resilience to domain shape alterations. CuPy GPU and PyTorch GPU maintain stable runtimes across most shapes, with minor variations occurring only at extreme aspect ratios where memory coalescing degrades slightly. JAX GPU demonstrates near-perfect insensitivity, keeping runtimes similar to the reference domain across all grid configurations. Overall, these findings confirm that domain shape must be factored in when evaluating CPU stencil performance due to cache layout effects, however, the effect is generally more pronounced for the CPU implementations than for the GPU implementations.

3.2. Field Validation

An exemplar selection of plots used for the visual validation of the diffusion application is shown in Figure B.1 to Figure B.7. While the results are shown only for one domain, similar plots were generated also for all other combinations of gridsizes. As one can see from them, the stencil code behaves as expected for all frameworks both with regards to the diffusion operator and the halo updates, and the results appear visually identical.

The numerical validation confirms that all implementations reproduce the Fortran reference result within the chosen absolute and relative tolerances.

3.3. Additional Diagnostics

Figure 3.4 shows the results of the benchmarking experiment with the number of threads fixed to one.

As expected, the performance of the GPU-based runs remains largely unaffected. In contrast, the runtimes of a few of the CPU-based implementations change considerably, particularly for larger working set sizes. This suggests that for these implementations performance benefits from some form of CPU-side parallel execution in the background, which becomes increasingly important as the problem size grows.

GT4Py exhibits a substantial slow down and quickly reaches a plateau at a runtime per grid cell of about $3 \cdot 10^{-6}$ s. By comparison, in the benchmarking run without fixed threading, it achieves runtimes per grid cell of less than 10^{-7} s (see Figure 3.1). The PyTorch CPU curve also changes noticeably: the performance recovery previously observed at larger working-set sizes disappears, and its runtime per grid cell flattens into a regime similar to NumPy. This behavior confirms that PyTorch relies heavily on multi-threaded parallel execution (via OpenMP) to amortize CPU overheads for larger domain sizes. Restricting execution to a single thread disables this background parallelization, preventing the framework from scaling efficiently as memory demands grow

The stencil code performance of other frameworks, like NumPy, Numba and JAX, seem to be insensitive to the the setting `OMP_NUM_THREADS=1`. This suggests either that these implementations are already effectively single-threaded for this workload or that their parallelization mechanisms do not rely on OpenMP but on some other backends.

3. Results & Discussion

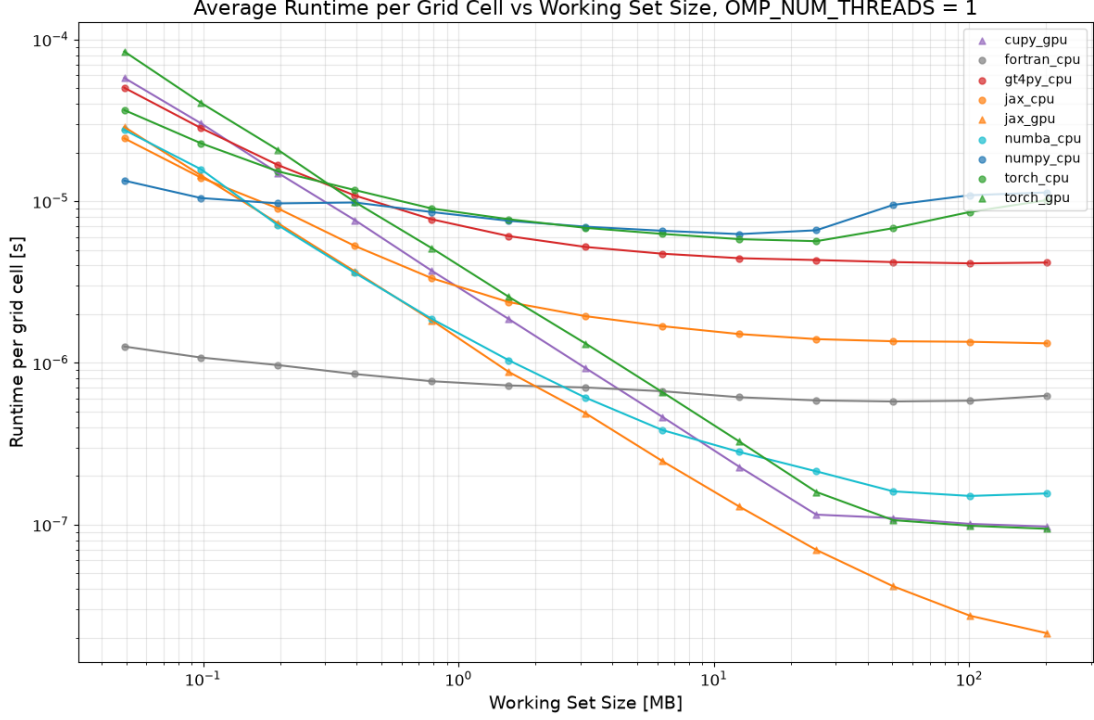


Figure 3.4.: Runtimes per grid cell as a function of the working set size, analogously to Figure 3.1 but with the number of OpenMP threads fixed to one via `export OMP_NUM_THREADS=1`.

The results of the runtime comparison between executions performed with and without the `master.py` script, shown in Table 3.1, suggest that the benchmarking framework does not introduce a substantial overhead. Most manually recorded runtimes fall within the range of runtimes observed across the ten runs executed via the script. The three outliers (Numpy and PyTorch on both CPU and GPU) lie only slightly outside this range and would likely fall within it if additional runs were performed. We therefore conclude that the benchmarking strategy did not significantly influence the measured performance and that the reported runtimes are representative of the actual execution times.

3. Results & Discussion

Framework	Manual Run [s]	<code>master.py</code> , runtime range after 10 runs [s]
NumPy	1.881628	[1.771330, 1.870093]
Numba	0.1319145	[0.129751, 0.135514]
JAX (CPU)	0.521480	[0.517538, 0.526609]
JAX (GPU)	0.123688	[0.123165, 0.128426]
PyTorch (CPU)	5.926811	[6.030340, 10.487709]
PyTorch (GPU)	0.297009	[0.297491, 0.403963]
GT4Py (CPU)	0.426039	[0.419773, 0.447851]
CuPy (GPU)	0.233636	[0.209686, 0.261795]

Table 3.1.: Comparison of runtimes obtained through direct execution and via the benchmarking framework for the cubic domain with 64 gridcells per side. The values are approximated to the sixth floating point digit.

3.4. Implementation Experience

To adapt the baseline NumPy code to GT4Py, a total of 122 lines of code had to be removed or added. Additionally, the Laplacian operator and the diffusion function had to be rewritten as a `@gtx.field_operator` and a `@gtx.program` respectively, as described in Table 3.2. Furthermore, a complete rewrite was required to introduce GT4Py’s dimension objects, as well as to implement both CPU and GPU backends. These changes were not straightforward and required a sufficient understanding of GT4Py and its programming model. Additionally, we struggled with porting the GT4Py code to GPU, and the error messages were at times difficult to interpret and debug.

In contrast, the CuPy implementation required changes to only 11 lines of code. Since CuPy provides an interface that is very similar to NumPy, the required modifications were relatively simple.

Numba and PyTorch required a similar number of changes with 43 and 44 modified lines, respectively. However, Numba required the explicit use of `prange` loops, meaning that the vectorized NumPy slices had to be rewritten using explicit loops. Apart from this, the Numba implementation was relatively straightforward, as decorators could be used to accelerate the code and activate JIT-compilation. For PyTorch, both the Laplacian operator and halo updates could remain unchanged. The main modification consisted of replacing the NumPy arrays with tensors and adding the option to execute the computation on either CPU and GPU. This resulted in a relatively easy adaptation of the original implementation.

JAX required more extensive code modifications than the other frameworks because of its reliance

3. Results & Discussion

on functionally pure programming and immutable arrays. Nevertheless, the necessary changes were largely repetitive and therefore straightforward to implement, and switching between GPU and CPU was very easy and smooth.

Framework	Changed Code Lines	Laplacian Operator	Halo Update	Experience
CuPy	11	unchanged	unchanged	only a few changes from NumPy to CuPy (array module swap, explicit sync calls).
Numba	43	changed	changed	NumPy’s vectorized slices had to be unrolled into explicit <code>prange</code> loops, plus <code>@jit</code> decorators.
PyTorch	44	unchanged	unchanged	substituting NumPy arrays for tensors required few adaptations for GPU use.
JAX	59	changed	changed	array mutations had to become functional <code>.at[...].set()</code> updates, and the loop became a <code>lax.fori_loop</code> .
GT4Py	122	changed	unchanged	complete rewrite using GT4Py’s dimension objects and <code>@gtx.field_operator</code> or <code>@gtx.program</code> .

Table 3.2.: Comparison of framework adaptations from the baseline NumPy stencil implementation.

4. Conclusion

4.1. Performance

Overall the implementations that are able to leverage the GPU are notably faster compared to the ones that only work on CPU, when working on larger datasets. However, it is interesting to see that the JIT-compiled, CPU-based approaches implemented in Numba and GT4Py are not far of the simple GPU implementations of Cupy and Torch. Which is remarkable when thinking of the much higher processing power of the GPU for these tasks. Only the JAX GPU implementation shows a significant increase in performance compared to all the other implementations. This however comes at the cost of needing more modification in the code compared to the other approaches.

4.2. Choice of Framework / DSL

The “best” framework for a project depends on many different factors. In the following we broke the decision down into essentially two main questions: Firstly, is a (CUDA capable) GPU available on the system and secondly, how much additional time is available for implementing the new approach. These decisions with their recommended framework can be seen in tab. 4.1. If a (CUDA-)GPU is available the first and easiest thing to do is to rewrite the program to CuPy or PyTorch. This results in a big performance increase for very little additional work. If no GPU is available the easiest approach with a big performance gain is Numba. If future GPU support is critical the recommended framework is GT4Py, because it offers similar performance to Numba but would theoretically allow for GPU implementation however we did not manage to get that running. If the best performance is critical, more implementation time is worth it and a GPU is available the recommended framework is JAX because it offered the best performance in our tests. These recommendations give a rough idea which framework is best for which type of project.

5. Outlook

Work needed	GPU available	recommended Framework	Benefits
Minimal	Yes	CuPy, PyTorch	very easy numpy drop in approach
Low	No	Numba	reasonably easy and very good CPU performance
Moderate	No	GT4Py	more complex to set up compared to Numba but should allow for GPU implementation and more config options (eventhough we were not able to get it to run on GPU)
High	Yes	JAX	Fastest approach but also with the highest implementation cost

Table 4.1.: Which framework / DSL is recommended depending on the time and hardware available

5. Outlook

To deepen the understanding of the differences between the frameworks, further configuration options could be examined. With this additional performance could be gained. Another point this project hasn't looked at is the future of the examined frameworks as well as their update cycles. If a lot of work will be done on a library it might be worth to invest some time into it because performance will probably improve in the future. This could, e.g., be another argument for choosing GT4Py because it is still under heavy development. Further different stencils could be examined because they could introduce new memory-compute ratios, which might affect the performance of each framework differently. Lastly, even larger domain sizes could be analysed to find the plateau size for the JAX GPU implementation.

5. References

- [1] The JAX Authors. *JAX Quickstart Guide*. 2024. URL: https://docs.jax.dev/en/latest/notebooks/thinking_in_jax.html (visited on 08/29/2026).
- [2] CSCS. *Santis*. 2026. URL: <https://docs.cscs.ch/clusters/santis/> (visited on 09/12/2026).
- [3] CuPy Developers. *CuPy User Guide*. 2026. URL: https://docs.cupy.dev/en/stable/user_guide/index.html (visited on 09/02/2026).
- [4] Numba Developers. *Numba Documentation*. 2026. URL: <https://numba.readthedocs.io/en/stable/index.html> (visited on 09/10/2026).
- [5] NumPy Developers. *NumPy User Guide*. 2026. URL: <https://numpy.org/doc/stable/user/index.html> (visited on 08/31/2026).
- [6] Suraj Subramanian et al. *Learn the Basics*. 2021. URL: <https://docs.pytorch.org/tutorials/beginner/basics/intro.html> (visited on 08/31/2026).

A. Library Versions

The following table lists all Python packages installed in the computational environment used for the experiments, including their respective versions.

Table A.1.: Complete list of Python packages used in the computational environment.

Package	Version
aiohappyeyeballs	2.6.1
aiohttp	3.13.3
aiosignal	1.4.0
alembic	1.14.1
anyio	4.8.0
argon2-cffi	23.1.0
argon2-cffi-bindings	21.2.0
array-api-compat	1.15.0
arrow	1.3.0
asttokens	2.4.1
astunparse	1.6.3
async-generator	1.10
async-lru	2.0.4
attrs	26.1.0
babel	2.17.0
bash_kernel	0.10.0
beautifulsoup4	4.13.1
black	26.5.1
bleach	6.2.0
boltons	26.1.0
cachebox	5.2.3
cached-property	2.0.1
cachebox	5.2.3
certifi	2025.1.31
certipy	0.2.1
cffi	1.17.1

Continued on next page

References

Package	Version
charset-normalizer	3.4.1
click	8.4.2
colorama	0.4.6
comm	0.2.3
commonmark	0.9.1
contourpy	1.3.3
cryptography	44.0.0
cuda-bindings	13.3.1
cuda-pathfinder	1.5.6
cuda-toolkit	13.0.3.0
cupy-cuda13x	14.1.1
cycler	0.12.1
cytoolz	1.1.0
dace	2.0.0a5
debugpy	1.8.21
decorator	5.3.1
deepdiff	9.1.0
defusedxml	0.7.1
devtools	0.12.2
dill	0.4.1
docopt	0.6.2
executing	2.2.1
factory_boy	3.3.3
Faker	40.36.0
fastjsonschema	2.21.0
filelock	3.30.2
fonttools	4.63.0
fparsen	0.2.4
frozenset	1.8.0
fsspec	2026.6.0
greenlet	3.1.1
gridtools-cpp	2.3.9

Continued on next page

References

Package	Version
gt4py	1.1.10
h11	0.14.0
httpcore	1.0.7
httpx	0.28.1
idna	3.10
ipykernel	7.3.0
ipyparallel	9.2.0
ipython	9.16.1
Jinja2	3.1.6
json5	0.10.0
jsonpointer	3.0.0
jsonschema	4.23.0
jsonschema-specifications	2024.10.1
jupyter-events	0.12.0
jupyter-lsp	2.2.5
jupyter_client	8.9.1
jupyter_core	5.9.1
jupyter_server	2.15.0
jupyterlab	4.1.8
kiwisolver	1.5.0
lark	1.3.1
llvmlite	0.48.0
Mako	1.4.1
MarkupSafe	3.0.3
matplotlib	3.11.0
ml_dtypes	0.5.4
mpi4py	4.1.2
mpmath	1.3.0
multidict	6.7.1
networkx	3.6.1
ninja	1.13.0
numba	0.66.0

Continued on next page

References

Package	Version
numpy	2.4.6
nvidia-cublas	13.1.1.3
nvidia-cuda-crt	13.3.73
nvidia-cuda-cupti	13.0.85
nvidia-cuda-nvcc	13.3.73
nvidia-cuda-nvrtc	13.0.88
nvidia-cuda-runtime	13.0.96
nvidia-cudnn-cu13	9.20.0.48
nvidia-cufft	12.0.0.61
nvidia-cufile	1.15.1.6
nvidia-curand	10.4.0.35
nvidia-cusolver	12.0.4.66
nvidia-cusparselt-cu13	12.6.3.3
nvidia-cusparselt-cu13	0.8.1
nvidia-nccl-cu13	2.29.7
nvidia-nvjitlink	13.3.33
nvidia-nvshmem-cu13	3.4.5
nvidia-nvtx	13.0.85
opt_einsum	3.4.0
packaging	26.2
pandas	3.0.5
pillow	12.3.0
platformdirs	4.11.2
prometheus_client	0.21.1
psutil	7.2.2
pybind11	3.1.0
pycparser	2.22
Pygments	2.20.0
PyJWT	2.10.1
pynvml	12.0.0
pyparsing	3.3.2
python-dateutil	2.9.0.post0

Continued on next page

References

Package	Version
PyYAML	6.0.3
pyzmq	27.1.0
referencing	0.36.2
requests	2.32.3
rich	12.6.0
rpds-py	0.22.3
ruamel.yaml	0.18.10
scipy	1.18.0
six	1.17.0
sniffio	1.3.1
soupsieve	2.6
SQLAlchemy	1.4.52
sympy	1.14.0
tabulate	0.10.0
tornado	6.5.8
torch	2.13.0
tqdm	4.70.0
traitlets	5.16.1
triton	3.7.1
typer	0.7.0
typing_extensions	4.16.0
urllib3	2.3.0
wcwidth	0.8.2
websocket-client	1.8.0
xxhash	3.8.1
yaml	1.23.0

B. Additional Figures

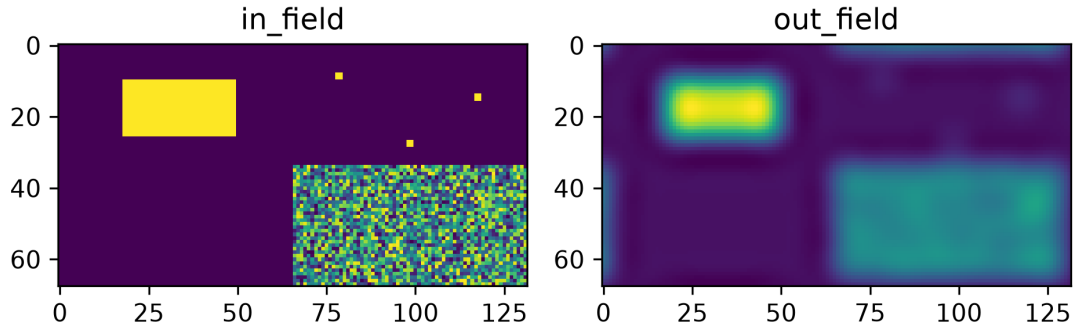


Figure B.1.: Fortran reference, for a domain with $n_x = 128, n_y = 64, n_z = 32$.

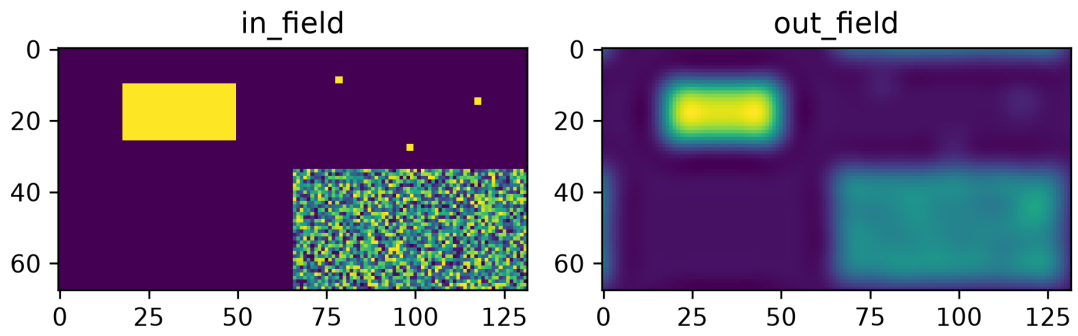


Figure B.2.: Numpy, same domain as in Figure B.1.

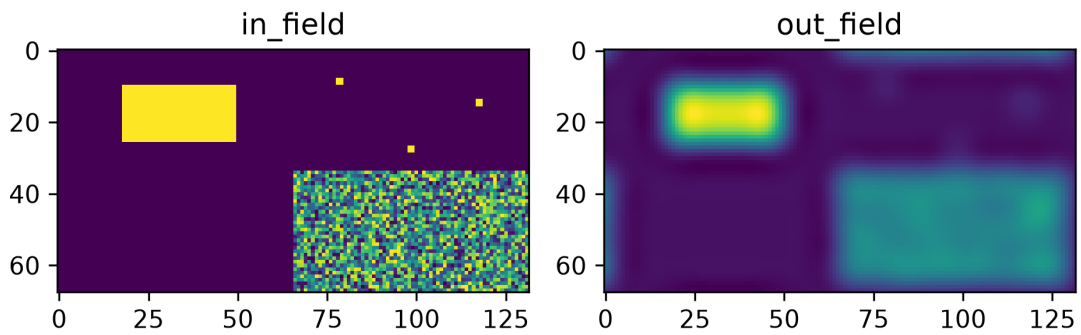


Figure B.3.: Numba, same domain as in Figure B.1.

B. Additional Figures

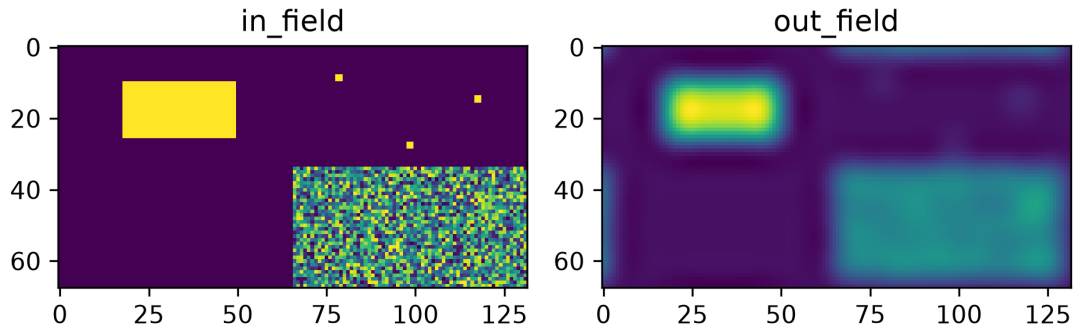


Figure B.4.: CuPy, same domain as in Figure B.1.

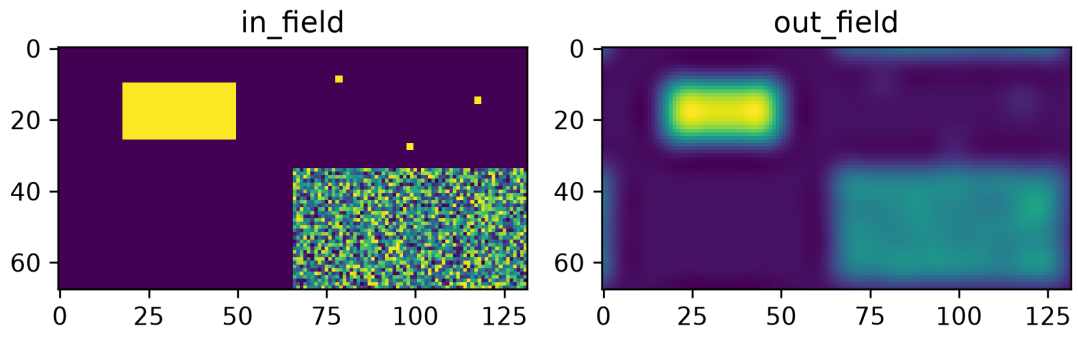


Figure B.5.: GT4Py, same domain as in Figure B.1.

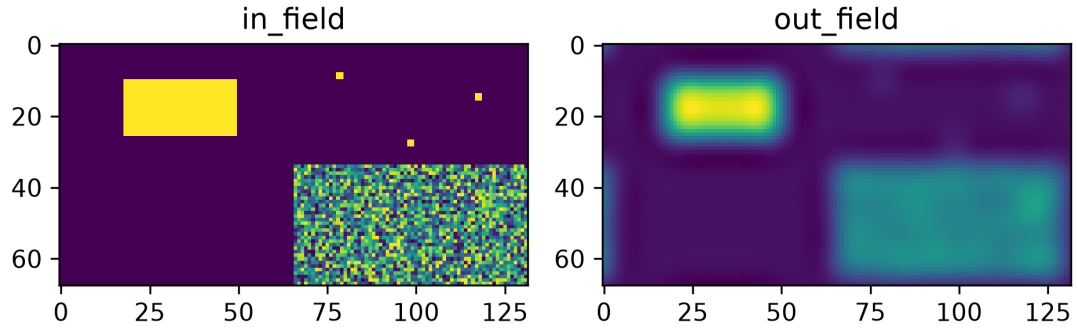


Figure B.6.: JAX, same domain as in Figure B.1.

B. Additional Figures

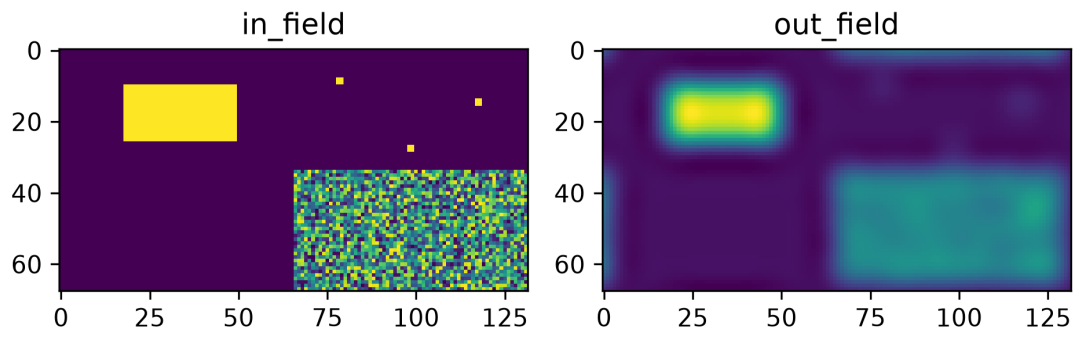


Figure B.7.: PyTorch, same domain as in Figure B.1.

C. Use of AI-Based Tools

Claude was used to assist in debugging the code and specifically to help develop the `stencil2d_fortran.py` wrapper, which enabled the execution and integration of the `stencil2d_fortran.F90` reference implementation within the `master.py` benchmarking workflow. M365 Copilot was employed to streamline the generation of the plots showed in Section 3, for grammar and stylistic improvements of the written text throughout the report, as well as for resolving syntax errors and formatting the LaTeX document.